ELEC 378 Spring 2026
Final Project Report

Austin Yilmaz, João de Lima Vana,Rishabh Rengarajan

1 Introduction and Team information

This report presents the challenges faced, how we overcame them, and our insights on the process for the final project of the ELEC378 course. Our task was to create a multiclass image classifier to classify butterfly and moth species and evaluate this classifier's accuracy on a Kaggle contest. We pursued several model architectures and made design choices that will be explained in detail below.

Our project team is named **The Front Row**; below is listed the team members and their contributions to the final project:

1. **Austin Yilmaz (ay87):**
   (a) Preparing the sectioning and outline for the project report
   (b) Coding a Convolutional Neural Network classifier training and evaluation pipeline through Python
   (c) Coding an SVM-based classifier training and evaluation pipeline through Python
   (d) Writing the model/workflow-related parts of the project report
2. **João de Lima Vana (jd185):**
   (a) Responsible for training and testing CNN locally
   (b) Developed a model for SIFT with KNN model for the non neural network model
   (c) Developed pre-processing techniques for data (evaluated different packages for data processing: e.g. background removal)
   (d) Developed "post-processing" technique
3. **Rishabh Rengarajan (rr150):**
   (a) Set up Github repo for collaborative work
   (b) Did initial (more naive) work on a SVM-based classifier model
   (c) Responsible for training/testing CNN models locally
   (d) Implemented pytorch-lightning into the CNN classifier for better visualization/cleaner code

2 Data exploration

2.1 General appearance of the dataset

At a high level, the dataset consisted of 12594 labeled images for the training set, and 1000 unlabeled images for the testing set. Both datasets contained images of 100 different classes, one class for each image (these classes being the particular species). In the training set, each class was associated with an average of 126 samples, with a minimum of 100 images and a maximum of 187 images per class, as pictured below in figure 1:

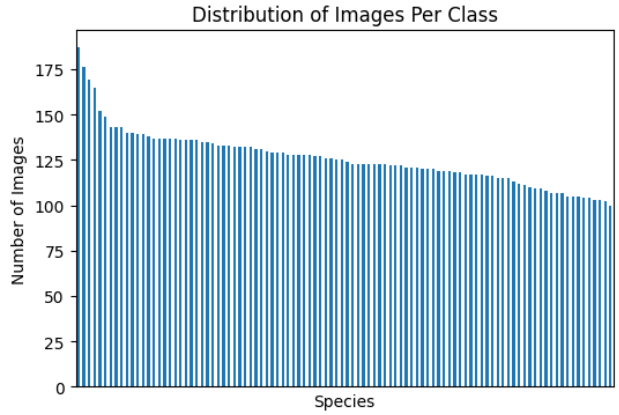


Figure 1: Distribution of species in training dataset

Furthermore, while each image was exactly 224x224x3 pixels (since they were RGB), we noticed a large variation in the information within images. More saturated images had a higher file size while simpler images (for example, white backgrounds) had lower file sizes. The largest pictures contained 50kB of data, and the smallest contained 9kB, while most of the images contained 20 to 30 kB, as shown in the plot below:

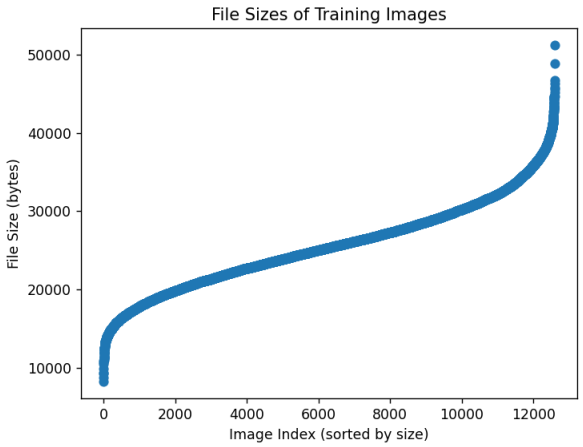


Figure 2: Distribution of image file sizes

In terms of visual variability, we noticed two broad categories of images: insects in a nature scene, or insects in an expository position (typically a white background, fully open wings, and front-facing). Before doing any work, we hypothesized that the exposition-pose images would be generally easier to classify due to the obvious location and clarity of the subject, while natural images could be potentially difficult due to flowers, grass, and other objects in the photograph other than the subject. For example, consider the following examples:

the following paragraph is sketchy

The exposition pose is the easiest one in terms of preprocessing (as our background removal packages excel at removing single colored backgrounds). Meanwhile, the natural ones posed a bigger challenge, as some pictures were overexposed (e.g. test_image 00999), some were on top of flowers (e.g. train_image 000199), which made edge detection harder. Also, we have realized that **color** and **wing shape** were the most drastic differentiators between two species of butterflies, so we incorporated the intuition we gained from those differences into the approaches we used in the preprocessing stage of our data.

(a) An overexposed image, potentially hard to classify (`test_000999.jpg`).

(b) A saturated image with flowers, potentially hard to classify (`train_000199.jpg`).

(c) An image with a clear exposition pose (clear subject, white background), likely easier to classify (`train_000187.jpg`).

2.2 How we split the dataset for training and validation

A common technique to ensure that we can correctly evaluate our model's performance and tune hyperparameters accordingly is to train the model on a training set and, evaluate it on a validation set. To reduce biases caused by the particular choice of the training/validation split, we also used k-fold cross validation (with k=5), in which we repeat the training/validation process 5 times with different splits and average the results to get a more stable estimate of our model's performance. We chose k=5 because we used the typical 80/20 train/validation split (80% of the data for training and 20% for validation) to maximize the number of images we can use for training while still having a good amount of data for validation; with that split, 5 folds would allow us to use all of our data for training and validation across the different folds.

reconsider this following part

We chose this approach to partitioning the data because it has been empirically shown to excel in independent and identically distributed (i.i.d.) datasets with dozens of classes, but later, we also considered the following additions to our split:

1. Since the butterfly data is not fully i.i.d because of the general appearance of the dataset we described above, and also we have relatively few images per species in the dataset, we can first ensure that each species is evenly weighted in the train/validation sets before training (by adding a term to account for how represented in the dataset each species is).
2. Since we found out that some images were **exposed**, and some were **natural**, we can **tag the images accordingly and ensure that both are represented in the train/validation sets before training**.
3. Another idea we found out that performed relatively well compared to naive approaches was training and validating the classifier was to use all data both to train and validate the model, but we thought that in relatively small per-class image numbers like our dataset, this would lead to overfitting; so we did not go into this approach.

3 Preprocessing of Data and Design Choices

3.1 Preprocessing pipeline applied to data before training

We chose to apply different preprocessing pipelines to our different model architectures, as they have different requirements and are affected differently by the preprocessing steps.

We started the preprocessing our data by resizing the images to 224x224 for our CNN model, 64x64 for SVM and converted them to NumPy/OpenCV arrays, which ensures that the sizes are standardized and makes conversion to/from data vectors/matrices easier, helping us produce a cleaner pipeline.

3.1.1 Preprocessing for the CNN Classifier

All of the transforms we used in our CNN pipelines were from the `torchvision.transforms` package, and we applied the following steps in our training pipeline:

1. `ToImage()`: convert the initial 224x224x3 image vector to a PyTorch image tensor, which allowed us to utilize CUDA and improve training/prediction speed.
2. `Resize(224 * 1.1)`: upscale the image tensor to about 245 pixels to create some room for randomness and thus help prevent overfitting to exact framings of a picture. (Professor Baraniuk demonstrated similar methods in class.)
3. `RandomCrop(224)`: randomly crop the image to its original size (224x224), to force our model to learn to identify an object "anywhere" in the image, not just in the center.
4. `RandomHorizontalFlip(p=0.5)`: since butterflies/moths are often symmetrically shaped, sometimes horizontally flip the image to prevent the model from having a bias toward left/right facing butterflies.
5. `RandomRotation(degrees=10)`: a butterfly/moth can also be arbitrarily oriented, so randomly rotate the image anywhere from -10° to 10° to prevent the model from overfitting to the vertical orientations of our butterflies.
6. `RandomAffine(degrees=0, translate=(0.1, 0.1))`: since the butterfly/moth can also be arbitrarily translated in the image, randomly translate the image by up to 10% of its size vertically and horizontally to prevent the model from overfitting to the centered position of our butterflies (for example, in the exposition pose images like figure 3c).
7. `RandAugment(ops=2, magnitude=10)`: randomly augment our pictures by changing their brightness/warmth/contrast/posing using . Our reasoning was that this would account for possible photographic editing (those are the most commonly adjusted parameters in phtographic softwares, such as Lightroom). Moreover, this also effectively simulates a real world variation since not all real butterfly pictures have the same brightness, and this step prevented overfitting to the few images of a specific butterfly species we had in our dataset.

   re-explain this probably
8. `ToDtype(torch.float32, scale=True)`: convert the pixel values to floats between 0 and 1.0, which is a more stable input range for our model to learn from, especially with ReLU activations.
9. `ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.02)`: we wanted our model to tolerate and learn different shades of a colored butterfly/genetic variations/dirt etc., but not too much since butterflies are mainly distinguished by their color, so apply a small random variation to the hue, saturation, saturation, and contrast of the image.
10. `Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))`: center and normalize pixel values to ensure that the input data works well with ReLU steps in our CNN and that the training produces more stable gradients.

    For our validation and test pipeline, we only convert the data to a Pytorch image tensor, ensure the image is 224x224x3, center and normalize, and convert to a float tensor. We do this because all the augmentation steps described above are useful in training the model, but we want consistent/stable validation, so we decided not to apply those preprocessing steps to our validation/test sets.

3.1.2 Preprocessing for HOG+RBF SVM

review this for accuracy

1. We used a smaller image size here (64x64), since we were not be able to parallelize because of the CPU-heavy inefficient nature of scikit-learn SVM algorithms, even with the addition of PCA.
2. We also did a horizontal flip here using `np.fliplr()`, exploiting the symmetry of butterflies, and preventing our model from overfitting to left/right facing orientations.
3. We then used a HOG-based edge/feature detector, which is a more modern and robust edge detection approach to the ones we learned in the class (like SIFT), using the method `hog()`. It essentially extracts edges, textures, and other shapes of the picture by computing gradients and comparing their directions.

This was because the wing shapes play a huge role in differentiating butterflies, as we observed from our dataset.

4. But one thing we skipped over when talking about HOG above was, it **skips coloring differences**. And as we mentioned, colors also play a huge role in differentiating butterflies, so we used histograms using `np.histogram(bins=32)` to count the red, green, and blue pixel intensities into 32 bins to append 96 features hinting about the color of our butterfly to our data vectors. This way, our model can differentiate between butterflies much more accurately.
5. To avoid brightness/natural noise in colors, just like in the CNN but with a slightly different approach, we normalized our color histogram values so that the color features are actually comparable between pictures.
6. We then unsurprisingly concatenated the HOG features with the color features by appending the color histograms to the HOG-based data vectors.
7. After that, we applied the **most important preprocessing step in the SVM pipeline**, which was to normalize and center all of our data vectors, because SVM (especially with RBF kernel) can be extremely sensitive to disproportionate deviations in features.
8. Finally, since HOG usually produces very large data vectors, we applied PCA to project all our data vectors onto the first n eigenvectors that would capture 95% of the variance so that both training and validation speed improve.
9. **The validation/test pipeline was the exact same except that we did not flip the images, which would just introduce randomness into prediction. We wanted our prediction to be predictable.**

3.2 Background removal

We hypothesized that, due to the extra background information in the natural images (e.g. flowers and grass), a model could potentially perform better if we removed that background information and gave the model only subjects to learn. In other words, we thought that converting all images into "exposition pose" (subject on plain background) would improve our classifier's accuracy. This segmentation task itself proved to be challenging, and we tried a few methods with OpenCV functions, the most effective of which utilizes the GrabCut image segmentation algorithm using a full-image initial bounding box as well as morphological erosion and dilation operations. After trying different user-chosen parameters with the GrabCut algorithm, This was generally successful at isolating the subjects, but sometimes removed too much from the subject or did not remove enough. Consider the examples of the original image and the background-removed image in Figure 4 below:

This technique improved the accuracy of our KNN+SIFT model a little, but surprisingly led to worse performance in our more complex CNN model. Presumably this occurs because it removes noise from the data, which KNN is more vulnerable to but a CNN can learn.

review this statement

3.3 Comparison of the used preprocessing pipelines

review

3.3.1 Feature extraction

1. **In our HOG+SVM model,** we deterministically computed edges using HOG and color distribution using histograms. This gave us a fixed representation of each image.
2. **In our CNN model,** we let the neural network 'learn' the features by detecting the edges in the early layers, the textures in the middle layers, the object parts in the deeper layers. This gave us a more adaptive representation of each image, provably superior to HOG+SVM since if that fixed representation misses something about the butterfly, HOG cannot recover, while CNN can learn more about the useful features by introducing layers.

(a) `test _00003.jpg`, original.

(b) `test _00003.jpg`, cropped; perfect segmentation of the subject and background.

(c) `test _00081.jpg`, original

(d) `test _00081.jpg`, cropped; failure to fully remove the flowers.

Figure 4: Original images and background-removed images

3.3.2 Data augmentation

1. **In our HOG+SVM model,** we only did a horizontal flip on the images. This is because edges do not change much by color/lighting changes and since HOG is a much lower capacity model (compared to neural networks), too much augmentation can drastically reduce model accuracy rather than improving it.
2. **In our CNN model,** we applied a bunch of different random augmentations on our data (as mentioned above), ranging from cropping it to changing the contrasts. This is because even though CNNs learn the features of an image well, they might sometimes learn them *too well*. We need a lot of variation in augmentation to prevent overfitting to the data, especially in a dataset like this where in many of the images the object (butterfly) is relatively in the center of the picture.

3.3.3 Local spatial information

1. **In our HOG+SVM model,** the preprocessing steps (HOG+color histograms) do not really tell us about the locations of a color in an image, which means the loses the spatial information while training. In non-neural-network pipelines, especially a HOG-based SVM, this is inevitable because HOG cannot differentiate colors, only shape and separate preprocessing approaches like color histograms only differentiate color, not shape.
2. **In our CNN model,** we use *convolutions* to keep and propagate the information on which color appears where in the image. Unlike the SVM approach, this helps us pinpoint what each part of a butterfly is colored and how it differentiates it from the others.

3.3.4 Normalization

1. **In our HOG+SVM model,** we normalize the features of our data *after* we are done with extracting the features. This is because HOG representations make the RBF kernel SVM extremely vulnerable to small disproportionate variations in features.
2. **In our CNN model,** we normalize the pixels of our data *before* we start learning the features. This is because the CNN needs a stable input range to learn the features and is much more stable to small variations compared to other approaches like the SVM.

3.3.5 Dimensionality reduction

1. **In our HOG+SVM model,** we apply PCA on our data vectors to capture 0.95 of the variance, because training and testing the SVM is a sequential, CPU-heavy process.
2. **In our CNN model,** we do not externally apply any dimensionality reduction but let the layers of our neural network handle it. The pooling, convolutions, and compression inside the hidden layers do the 'dimensionality reduction' for us. Moreover, CNN training is parallelizable with CUDA in nature, so it is much more efficient with more features.

4 Models Considered for the Project

4.1 Model 1: ResNet CNN

4.1.1 Why we chose to try it

We have read some articles online that the peak image classification accuracies in diverse datasets like Caltech 101 were around 70-80% without the use of convolutional neural networks (CNNs), and that CNNs broke this wall and reached around 90-99% accuracy with ease. We also checked our homework description that said **at least one model should NOT involve a neural network**, and took the hint that neural networks are probably some good classifiers. .

add citations for these claims

4.1.2 Algorithm/Workflow

A convolutional neural network performs similarly to how a human brain is thought to visually recognize an image, and it has a simple pipeline once the algorithm is understood well. It works by passing the inputs through a variety of hidden layers before passing to a 'logistic regression' output layer. Moreover, it uses matrix convolution rather than explicit vector/array multiplication to significantly reduce the number of model parameters, allowing us to deepen the model without significantly decreased performance or vanishing gradients in the backpropagation process. The workflow is as follows:

1. We assume our images have been preprocessed and input to the CNN to train the model.
2. In the first layer, we first perform **feature extraction** on the input using 2-dimensional convolutions to learn the edges and color gradients.
3. We then downsample the data using max-pooling so that we only keep the strongest indicators/features.
4. Then, we pass them to the middle hidden layers, which build up increasingly complex patterns from the features using an arbitrary/tuneable variety of approaches where we will discuss in the *How Each Model Was Trained* section.
5. After that, we perform a spatial compression using average-pooling, which removes the spatial dimension to only leave the information about 'how strongly each feature was present in the data'.
6. Finally, we flatten our data to a vector that is the size of the number of our classes, similarly to how one would do 'logistic regression' in the final layer of our network.
7. Now that we are ready to score our vector, we optimize over the cross-entropy loss function we learned in class, with a good optimization algorithm (adamW, for example).
8. We then backpropagate the gradients we get from our optimization algorithm, and update the weights of our model.
9. We repeat this process until our loss converges, or until we hit the maximum number of epochs we set.

4.1.3 Design choices that affect the model

1. **Amount of feature complexity through layers:** We want to gradually increase feature complexity through the layers of our neural network, so in the first layers, we want to learn simpler things like edges and color; but as we go deeper in the network, we also want to learn more specific things like the markings on a butterfly. So, we need to gradually increase feature sizes and complexity of the operations we perform in the layers and residual blocks. (for example, start with just linear layer + relu, and continue with multiple layers of relu + 1-norm normalization)
2. **How much to reduce image size as we process it:** We want easier to analyze, smaller representations of an image that preserves what is present in the image correctly. But we also do not want to lose the actual features of an image that differentiate it from the others. So, we need to choose the step by step scaling accordingly.
3. **The number of residual blocks and layers we use:** We want a model that both accurately detects objects (butterflies) and has high performance in training, validation and testing, also speed-wise. Therefore, we need to be careful about adjusting how many features each residual block introduces into the data, and how many of the layers/blocks we have. Too many layers and residual blocks can impact performance while introducing marginal accuracy/maybe causing overfitting, but too few can cause the model to underfit or become unstable.
4. **Learning rate: scheduled or constant?** A learning rate too large can cause the model to become very unstable or overshoot the optimum, but one too slow can negatively impact the learning performance of our model. Hence, we should decide on what the initial learning rate is, and if we should decay the learning rate or not.

5. **How to decay the model weights over time:** Too large model weights can cause the model to overfit, and too much decay in the weights can cause the model to underfit. We also need to implement a schedule on decaying the weights of our model to keep it consistent.
6. **What to choose the batch size:** Since the optimization methods we use are usually based on batch optimization (like adamW), we need to choose a batch size that is enough to produce a stable training process, but one that is not too large so that the training stays fast.
7. **What to choose the loss function and optimizer:** Like in many other problems, the cross-entropy loss function is generally unmatched in its success here. So, we do not regard it as that flexible of a choice but still wanted to include it here. Coming to the optimization algorithm, we need an optimization algorithm that both incorporates fast early learning, and later fine-tuning. Moreover, it would help us if our optimizer decayed the weights/scheduled the learning for us. (Yes, we are hinting at adamW, adaptive moment estimation with weight decay)
8. **The preprocessing steps before the data is input: This is in fact the most important part as CNNs depend heavily on their input, just like most classification models. We have already discussed these choices and the steps we used in the previous section.**

4.1.4 Course concepts used in the model

1. **Supervised learning/Classification:** Obviously, the CNN we trained here is effectively a supervised learning model where inputs are images (data) with labels and the outputs are a discrete number of classes. Hence, this is effectively a classification problem.
2. **Feature engineering and learning:** Our CNN learns the features of the data by applying convolution layers and residual blocks.
3. **Optimization/OR:** We have a convex loss function that is called cross-entropy (log-loss), and we use an optimization algorithm based on *Gradient Descent* that is called adamW.
4. **Linear algebra background:** Since we are working with matrices, tensor operations, convolutional layers, max-pooling, and backpropagation of gradients; we are especially counting on the concepts we learned from Multivariate Calculus and Linear Algebra.

4.1.5 Loss function of the model and how it is optimized

As we mentioned, our model uses the cross-entropy loss, otherwise known as log-loss; and uses adamW to optimize it. The cross-entropy loss is derived from Maximum Likelihood Estimation (MLE) of logistic regression, and it has two terms, where one activates and the other deactivates. It is also convex, and hence optimizeable by gradient descent-based algorithms. AdamW is a fairly new algorithm that uses 'adaptive moment estimation' and 'weight decaying', in which the 'moments' are closely related to gradients in gradient descent; and weight decaying is just a smart way of steering the model away from underfitting to more previous data in the training procedure.

4.2 Model 2: HOG+RBF Kernel SVM

4.2.1 Why we chose to try it

It was given to as our Homework 7 in the course. The homework problem was about implementing a Kernel SVM using the Gaussian (RBF) kernel and fitting it to the butterfly data we have at hand and finally, submitting it to Kaggle. We also chose this model to have a strong contender for the required 'non-neural-network based model' in the final project. We read some articles online that suggested with the right training regimen and preprocessing (HOG), kernel SVMs can achieve accuracies as high as 70% in multi-class classification problems similar to our dataset (like Caltech 101).

4.2.2 Algorithm/Workflow

Support vector machines (SVMs) are essentially classifiers that fit an arbitary-dimensional hyperplane to separate the data points. But moreover, they try to 'regularize' that hyperplane by actually fitting a hyper-slab; achieving that by maximizing the margin of the hyperplane (thickness of the slab). This ensures that the classification model is more stable for the future data to come, even if it trades off some accuracy for the training data while doing so. And most of the time, the data we are working with is not 'linearly' separable in d dimensions, so we use one of the most powerful kernels that we can apply the kernel trick on, which is the RBF kernel; because any data is 'linearly' separable in ∞ dimensions, which the RBF kernel maps the data to. With the combination of these two approaches, the Kernel SVM works as follows:

1. We assume all the preprocessing steps (up until PCA) have been applied to the data before it is input to the Kernel SVM for training.
2. We first use the Gaussian (RBF) kernel to map our data to 'infinite' dimensions.
3. We then using the kernel trick, optimize our loss function (which is the '1/margin'-penalized hinge-loss and train the SVM.
4. While training, depending on $\lambda \sim 1/C$, we let some of the points be misclassified, for which we call them our **support vectors**.
5. We then use the support vectors to implement our decision function, in which the decision function looks like as follows:
$$f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$
6. Depending on the decision function value, we predict the class of the validation/test set. i.e.:
$$\hat{y} = \arg\max_k \ f_k(\mathbf{x})$$

4.2.3 Design choices that affect the model

1. **Kernel choice:** We need a kernel that has enough dimensionality to get us a linearly separable transformation, but also one that prevents overfitting and is stable. RBF kernel is known to have both of those features. For example, the sigmoid(tanh) kernel is known to be unstable even though it also maps the data to 'infinite' dimensions; and the polynomial kernels are known to overfit/underfit easily to complex datasets.
2. **Regularization parameter C:** Do we want higher bias, lower variance (higher C) in our hyper-slab, or vice versa (lower C)? This can drastically change our prediction accuracy, and is a good 'knob' to adjust to get higher prediction accuracy/lower overfitting levels.
3. **RBF exponentiation scaler γ:** If we increase this value, even the slightest spatial local adjustments between data can affect our classifier, which can lead to overfitting. However, if we decrease this value, our classifier may underfit. We also need to pay attention to this 'knob' to get a good classifier.
4. **Just like the CNN model, the preprocessing steps before the data is input are the most important of the design choices we can make, as most of the work we have done on training this classifier was actually in the preprocessing part (HOG+PCA). If the input data does not provide a well representation of the images we are trying to classify, what good is optimizing over it?**
5. **Course concepts used in the model**
   (a) **Supervised learning/Classifiaction:** Similarly to the CNN model, we are essentially doing supervised learning here, and the problem is a classification problem.
   (b) **Optimization/OR:** We are solving a convex optimization problem by minimizing hinge-loss while maximizing margin in the SVM training. We are using a variant of (stochastic) gradient descent to solve this problem, hence we are relying on our optimization principles.
   (c) **Unsupervised learning/PCA:** This is only used in the preprocessing step to make our training process smoother and faster.

(d) **Feature engineering and learning/HOG+color histograms:** While this is also only used in the preprocessing step, this part is very crucial to our model and we need a good manual design of features to accurately represent our data.

(e) **Kernel methods/Kernel trick:** This is the core method to make our data vectors linearly separable, and we achieve this using the RBF kernel + the kernel trick.

(f) **Linear algebra background:** We are also working with matrices and vector operations here both in the preprocessing of the data and the training of the model. We also use gradients in optimization, hence Multivariate Calculus and Linear Algebra also heavily appear in this model.

4.3 Model 3: KNN + SIFT

When we started looking for filters for our image processing step, we noticed that SIFT reduced each image from nearly tens of thousands of parameters down to 196. Then, as we remembered that the one of the main issues with KNN was the time complexity we thought that given the reduced data set size, it would be more manageable for it.
Furthermore, as the points of interest are scale and rotation invariant, this improves KNN ability to deal with rotated pictures (which was a severe issue as it changes almost all the pixels around).
In the end, our performance was at most around 42 % for this model. So, it was still not the best non-neural network based model (which was rather unfortunate, given this was our original model)
We initially were considering other ways to improve our model. However, we ran into a few issues, the most important one is that SIFT does not account for any form of color, which is quite helpful for classifying butterflies. Moreover, when we started looking into other possible filters, almost all of them increased the image complexity- which made KNN very inefficient at classifying. Thus, given that the performance (at best around 40%) was still considerably worse than the our simple first implementation of SVM, we ended up deciding not to proceed any further with this architecture.

5 How Each Model Was Trained

5.1 Model 1: Neural Network

TODO: Elaborate on how the model training and validation procedure works/worked. Include the design choices you made overtime (from the design choices you listed above that affect the model) and why you made those choices. If you made any changes to those choices or the procedure, list them and explain why you made the change.

5.2 Model 2: Whatever

6 Quantitative Comparison of the Models

6.1 Model 1: Neural Network

6.1.1 Training performance and robustness

TODO: Explain how RELATIVELY easy it was to train the model with the data at hand.

Although training the ResNet was the most computationally expensive part of the entire project, it was still a relatively easy process. As the architecture is extremely flexible in learning patterns, and we utilized different packages to optimize for the hyperparameters (mention a few ones, e.g. ADAM, or the one Rishabh used for early stopping), we did not have to account so much for particular aspects of our data (at least not compared to when we implemented the other methods, that lead us to HOG and SIFT).

6.1.2 Validation performance and accuracy

TODO: Provide data on how accurately the model validated the validation set.

6.1.3 Kaggle/test accuracy of the model

As seen in our Kaggle submission, we managed to obtain an extremely high prediction accuracy, which was comparable to the one from our test dataset. **TODO: Elaborate on the Kaggle accuracy of the model, and how it compares to the validation accuracy.**

6.1.4 Relative strengths and weaknesses of the model

TODO: Based on how the model quantitatively compares to the other models in terms of performance, also based on the correspondence between its validation and test performance, elaborate on the strengths and weaknesses of the model. For example, a strength can be validation and test accuracies both being high, implying less overfitting. A weakness can be long training times and requirement of precise tuning.

6.2 Model 2: Whatever

do the same thing as above

7 Overfitting and Generalization Analysis

TODO: Describe the instances where your models overfit/underfit, by first stating which model it was, and elaborate on what caused it. Then discuss strategies you resorted to, to overcome that over/underfitting. You might find it useful to explain overfitting/underfitting using the validation/test accuracy discrepancies of the model stated above.

7.1 Overfitting in Neural Network Model

7.1.1 Evidence of overfitting

The main evidence of overfitting in our model, was that we had a training accuracy of over 99%, and a validation accuracy of 93 % (our final submission accuracy was somewhat higher, but we achieved that by using a voting model across our different data splits). So, this is the main trait of overfitting, improved performance on training set, which did not improve the actual model performance.

7.1.2 What may have caused it

As we have learned in class, because neural networks have such exceptionally large number of weights to classify. Thus, as expected, it manages to reduce the training data error significantly, even though it does not translate into actual improvements in the accuracy.

7.1.3 How we tried to overcome it

TODO: Elaborate on what methods you used on mitigating the overfitting, and how well it mitigated it. If you were unable to improve, relate it to model complexity and task difficulty.

We reduced overfitting by doing early stop, using weight decay (i.e. a ridge penalty), and doing dropout

7.2 Underfitting in whatever

same argument.

8 Pipeline for the Final Submission

TODO: Explain each process of an image's journey in the final submission pipeline, starting from its preprocessing up until how it's classified with a given label. HIGHLY RECOMMEND TO ADD A FLOWCHART OR A VISUAL DRAW.IO MODEL ETC HERE!

8.1 Preprocessing of the image

TODO: Briefly explain how a particular image is preprocessed in the final pipeline.

8.2 Final model choice and its properties

TODO: Briefly explain what model you chose for the final pipeline touching on the properties of the image that the model suits well.

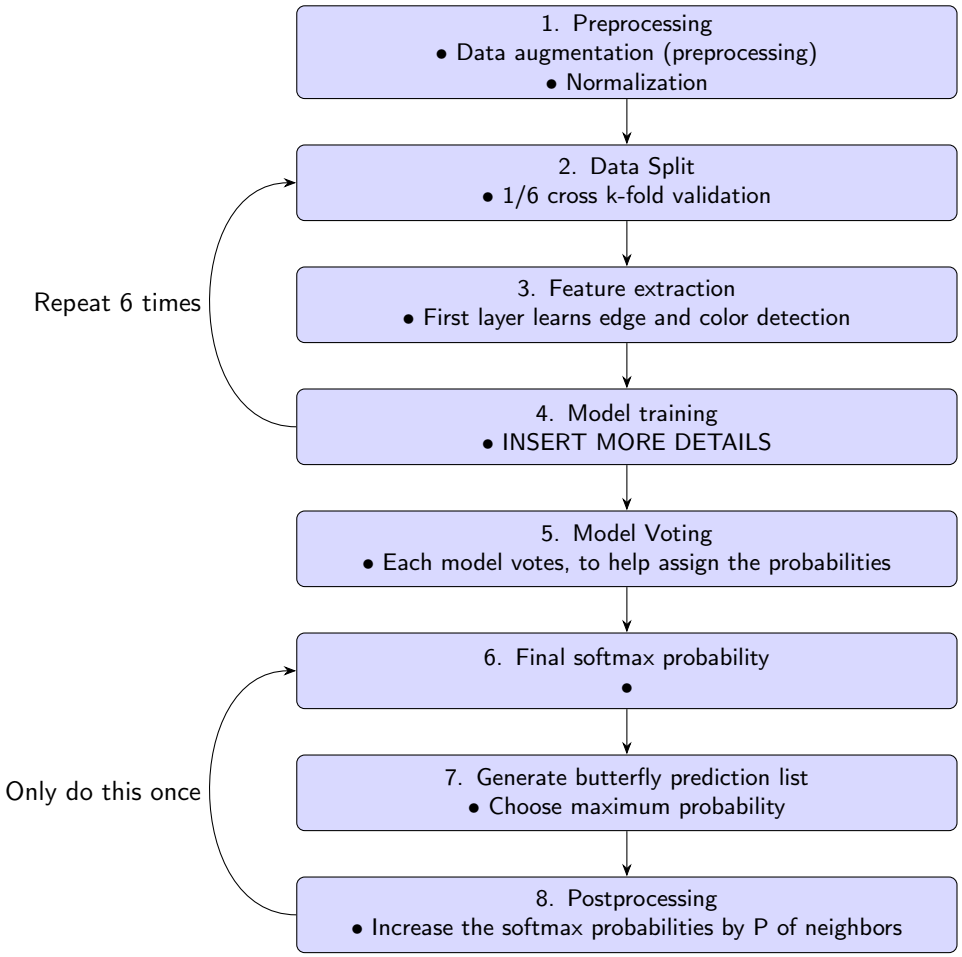
8.3 How the image trains the model

TODO: Briefly explain how the image is used in the training process of the model, touching on exactly how it improves the optimization process

8.4 How the model classifies the image

TODO: Briefly explain how the model processes the image to label it.

8.5 Flowchart



9 "Post-Processing"

(Note for grader: we asked on Piazza whether this is a valid technique, and the instructor response said it was)

After we looked at the test images, we noticed an odd pattern: there was significant cluttering of butterflies in sequential order. This makes sense, considering that if we were to look at a user feeding data into some form of database, they would probably in clusters- as someone might take multiple pictures of the same butterfly. Furthermore, it is expected that the classification would have some natural statistical biases, for example- we might expect that if a past butterfly is from a certain location, then other species in the same region would be more likely. Thus, it makes sense to also account for those patterns when creating a classifier.

Hence, we altered our prediction pipeline. Once we had our predicted list of butterflies, we retroactively scaled the softmax probability vector by a (1- $2P$) and added P (a tunable hyperparameter) to the probability of the neighboring butterflies. With that, we managed to achieve 99.8 % accuracy on the test dataset.

It is worth noting though that this also increases the chance of overfitting to the particular dataset that we were working with, if someone decided to shuffle the data the accuracy would then go down quite a lot; so, it is important to tune the P value.

10 Results and Conclusion

10.1 Results of the project - what we learned

TODO: Reflect on the overall results of the project and what new information you have learned.

10.2 Comments on the final model's performance

TODO: What do you think about how your final model performed in terms of performance and accuracy? Elaborate. We were reasonably satisfied with the final performance of our models. In particular, the ResNet was quite good at predicting, an

10.3 What surprised us

TODO: List all the things that were unexpected. They can be good, bad, or ugly. We were surprised when looking at some of the misclassified butterflies. While the majority of them had some large visual variation (e.g. the wings were shown from the top, which has a very different visual aspect to the bottom of the butterfly, and looked like another species). However, there some classifications that seemed similar to the rest of the dataset, so we were surprised that they were misclassified (which goes to show, as discussed in class that Neural Networks can be sensitive to small changes, producing unpredictable errors).

10.4 What's new to come